Pre-class Git practice

14.33 Research and Communication in Economics · Fall 2026

Complete this exercise before the Version Control and Reproducibility lecture.
Expected time: approximately 30 minutes.

Purpose

This exercise gives you concrete experience with the standard Git workflow:

```
pull → edit → inspect → stage → inspect → commit → push
```

You will also create and correct three safe mistakes. Each mistake will occur at a different point in the workflow.

Before you start

Complete the GitHub setup and course workflow first. You must have:

- a personal course repository named `fa26-1433-YOUR-KERBEROS`;
- a local clone of that repository; and
- Git configured with your name and email address.

This exercise uses the command line. You can continue to use GitHub Desktop for other course work.
A command-line **option** changes how a command operates. A long option starts with `--`, while a short option starts with one `-`.
For a visual explanation of commits and branches, see Git for Ages 4 and Up. The complete video is longer than this exercise, so you do not need to watch all of it now. You can also use the Missing Semester version-control notes as a reference.
Protect your existing work. Run each command only in your personal course repository. Do not use `git restore` if `git status` shows unrelated work that you want to keep.

1. Open the repository and inspect it

1. Open a terminal.
2. Change to your local course repository. Replace the example path with the actual path on your computer.

```
cd path/to/fa26-1433-YOUR-KERBEROS
```

3. Confirm that the terminal is in the correct repository.

```
git rev-parse --show-toplevel
git branch --show-current
git status
```

`--show-toplevel` prints the repository's top-level directory, and `--show-current` prints the current branch name.
The first command must show the path to your personal course repository. The second command should show `main`.
4. Check the output from `git status --short`.
`--short` tells Git to use a compact status format.

- If the command shows no changes, do not run the stash commands. Continue to step 5.
- If the command shows changes, store those changes temporarily with the following commands.

```
git stash push --include-untracked -m "Before Git practice exercise"
git stash list
git status
```

`--include-untracked` adds untracked files to the stash, and `-m` attaches the specified message to the stash.
`git stash` stores tracked and untracked changes outside the working tree. The files return at the end of this exercise.
The first entry from `git stash list` should include `Before Git practice exercise`. `git status` should now include `working tree clean`.
If you created a stash, make a note that you must restore it in section 7. If you did not create a stash, ignore all later stash instructions.
5. Download commits that exist on GitHub but not in your local repository.

```
git pull --ff-only
```

`--ff-only` permits the pull only when Git can update the branch without creating a merge commit.
If GitHub opens a browser for authentication, complete the browser prompts.
The command will report new commits or state that the repository is already up to date.

2. Create, stage, commit, and push a change

1. Open the repository in a text editor.
2. Create a file named `git-practice.md` in the repository's top-level directory.
3. Add the following text. Replace `YOUR-KERBEROS` with your MIT Kerberos username.

```
# Git practice

Repository: fa26-1433-YOUR-KERBEROS

I can inspect a change before I commit it.
```

4. Save the file.
5. Inspect which files are untracked, modified, or staged.

```
git status --short
```

`--short` tells Git to use a compact status format.
Git should report `?? git-practice.md`. The two question marks mean that Git does not track the file yet.
6. Stage the file for the next commit.

```
git add git-practice.md
```

7. Inspect the proposed commit.

```
git status --short
git diff --staged
```

`--short` requests compact status output, and `--staged` compares the staging area with the most recent commit.
Read the diff. Confirm that the file contains your Kerberos username and no sensitive information.
8. Record the staged snapshot as a commit.

```
git commit -m "Add Git practice notes"
```

`-m` supplies the commit message directly in the command.
9. Inspect the new commit and confirm that no uncommitted changes remain.

```
git log -1 --oneline
git status
```

`-1` limits the log to one commit, and `--oneline` displays that commit on one line.
10. Send the commit to GitHub.

```
git push
```

Your local repository and the GitHub repository now contain the commit.
Pause and reflect: Which command changed the staging area? Which command changed the local history? Which command changed GitHub?

3. Mistake 1: discard one uncommitted change

You will add an incorrect statement, inspect it, and discard it before staging it.
1. Add this line at the end of `git-practice.md`:

```
The baseline sample excludes all observations.
```

2. Save the file.
3. Inspect the change.

```
git status --short
git diff -- git-practice.md
```

`--short` requests compact status output, while the standalone `--` ends the options and identifies `git-practice.md` as a path.
The line is intentionally incorrect. You do not want to keep it.
4. Discard the uncommitted change to this file.

```
git restore git-practice.md
```

`git restore git-practice.md` permanently discards uncommitted changes in that file. The command does not affect other files when you specify only `git-practice.md`.
5. Verify the result.

```
git status --short
git diff -- git-practice.md
```

The standalone `--` again identifies `git-practice.md` as a path rather than an option.
Both commands should produce no output. Open the file and confirm that the incorrect line is gone.
Pause and reflect: Why did this correction not create a new commit?

4. Mistake 2: unstage a change

You will stage a correct change too early. You will then remove it from the proposed commit without deleting the change.

1. Add this line at the end of `git-practice.md`:

```
I can remove a change from the staging area without deleting the change.
```

2. Save and stage the file.

```
git add git-practice.md
git diff --staged
```

`--staged` compares the staging area with the most recent commit.
3. Remove the file from the staging area.

```
git restore --staged git-practice.md
```

Here, `--staged` makes `git restore` change the staging area instead of the working file.
4. Inspect both locations.

```
git status --short
git diff
git diff --staged
```

`--short` requests compact status output, and `--staged` limits the second diff to staged changes.
`git diff` should show the new line. `git diff --staged` should show nothing. Unstaging did not delete your work.
5. Stage the file again and record the change.

```
git add git-practice.md
git diff --staged
git commit -m "Document the staging workflow"
git push
```

`--staged` shows the proposed commit, and `-m` supplies the commit message.
Pause and reflect: After you unstaged the file, where did the new line remain?

5. Create and merge a branch

A branch gives a line of work its own name. You will make one commit on a new branch and then merge it into `main`.
1. Create a branch named `practice-branch` and switch to it.

```
git switch -c practice-branch
```

`-c` tells `git switch` to create the named branch before switching to it.
2. Confirm that `practice-branch` is the current branch.

```
git branch --show-current
```

`--show-current` prints the current branch name.
3. Add this line at the end of `git-practice.md`:

```
I can develop a change on a separate branch and merge it into main.
```

4. Save, inspect, stage, and commit the change.

```
git status --short
git diff
git add git-practice.md
git diff --staged
git commit -m "Add branch practice note"
```

`--short` requests compact status output, `--staged` shows the proposed commit, and `-m` supplies the commit message.
5. Inspect the branch's new commit.

```
git log -1 --oneline --decorate
```

`-1` limits the log to one commit, `--oneline` uses one line, and `--decorate` shows branch labels.
6. Switch to `main` and merge `practice-branch` into it.

```
git switch main
git merge practice-branch
```

Git will usually report a **fast-forward** merge here. Because `main` did not change, Git can move the `main` label to the branch's commit.
7. Inspect the merged history.

```
git log -2 --oneline --decorate
```

`-2` limits the log to two commits, `--oneline` uses one line per commit, and `--decorate` shows branch labels.
8. Delete the merged branch and push the updated `main` branch.

```
git branch -d practice-branch
git push
```

`-d` deletes the branch label only if Git confirms that its commit is already merged.
Delete a short-lived branch after you merge it. The commit stays in `main`; only the temporary branch label is removed.
Pause and reflect: Did merging create another commit, or did it move the `main` label to the branch's commit?

6. Mistake 3: reverse a pushed commit

You will publish an incorrect commit. You will then reverse it with a new commit. This process preserves the history on GitHub.
1. Add this line at the end of `git-practice.md`:

```
This sentence is intentionally wrong and should not remain in the file.
```

2. Save, stage, inspect, commit, and push the incorrect change.

```
git status --short
git diff
git add git-practice.md
git diff --staged
git commit -m "Add an incorrect practice statement"
git push
```

`--short` requests compact status output, `--staged` shows the proposed commit, and `-m` supplies the commit message.
3. Inspect the recent history.

```
git log -3 --oneline
```

`-3` limits the log to three commits, and `--oneline` displays each commit on one line.
The first entry is the incorrect commit. The commit now exists locally and on GitHub.
4. Reverse the most recent commit with a new commit.

```
git revert --no-edit HEAD
```

`--no-edit` accepts Git's automatic message for the reversal commit.
`git revert` does not erase the incorrect commit. It creates a new commit that reverses the earlier change.
5. Inspect the result.

```
git log -4 --oneline
git status
```

`-4` limits the log to four commits, and `--oneline` displays each commit on one line.
Open `git-practice.md`. Confirm that the intentionally wrong sentence is gone.
6. Push the reversal commit.

```
git push
```

Pause and reflect: Why did `git revert` add a commit instead of erasing the incorrect commit?

7. Verify your completed exercise

1. Run the final checks.

```
git status
git log -5 --oneline
```

`-5` limits the log to five commits, and `--oneline` displays each commit on one line.
Your final `git status` must include `working tree clean`.
Your recent history should include these five commits, with the newest commit first:

```
Revert "Add an incorrect practice statement"
Add an incorrect practice statement
Add branch practice note
Document the staging workflow
Add Git practice notes
```

2. Open your repository on GitHub.
3. Confirm that `git-practice.md` does not contain either incorrect sentence.
4. Confirm that GitHub shows the same recent commits.

You have completed the Git practice when the file and all five commits are visible on GitHub.
5. **Conditional:** Complete the remaining steps only if you created a stash in section 1. Otherwise, the exercise is complete.
6. Confirm that the stash still exists.

```
git stash list
```

The first entry should include `Before Git practice exercise`.

7. Restore the files that you stored in the stash.

```
git stash pop
git status
```

Your earlier changes should return to the working tree. `git status` can now show modified or untracked files. This result is correct because you restored your earlier work.

The stash steps at the beginning and end of this exercise are conditional. Do not run `git stash pop` unless you created the named stash in section 1.

Reflection before class

Before class, explain these distinctions in your own words. You do not need to submit your answers.

1. How are staging, committing, and pushing different?
2. What kind of mistake does each command correct: `git restore`, `git restore --staged`, and `git revert`?
3. Why should you inspect `git status` and the relevant diff before you stage, commit, discard, or reverse changes?
4. What changed when you merged `practice-branch` into `main`?

Command summary

Command	Purpose
`git pull --ff-only`	Download remote commits without creating an unexpected merge commit
`git stash push --include-untracked -m "message"`	Store tracked and untracked changes temporarily
`git stash pop`	Restore the most recent stash and remove it from the stash list
`git status`	Show the current branch and identify untracked, modified, and staged files
`git diff`	Show unstaged changes
`git add <file>`	Add file content to the staging area
`git diff --staged`	Show the proposed commit
`git commit -m "message"`	Record the staged snapshot
`git push`	Send local commits to GitHub
`git switch -c <branch>`	Create a branch and switch to it
`git switch <branch>`	Switch to an existing branch
`git merge <branch>`	Integrate the named branch into the current branch
`git branch -d <branch>`	Delete a branch label after its work is merged
`git restore <file>`	Discard uncommitted changes in one file
`git restore --staged <file>`	Unstage one file without deleting its changes
`git revert <commit>`	Create a new commit that reverses a prior commit

When you are uncertain, inspect the repository with `git status` and a relevant `git diff` before you stage, commit, discard, or reverse changes.